

5 RAG with Multiple Representations

Based on a presentation by Bryan Bischof and Ayush Chaurasia

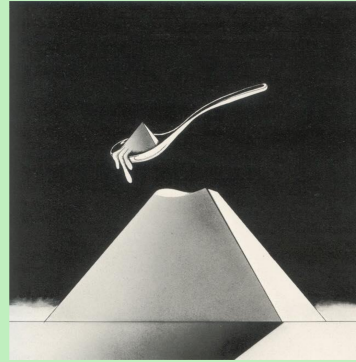
[Bryan Bischof](#) and [Ayush Chaurasia](#) argued that effective retrieval lies not in finding a single, perfect data representation, but in creating and leveraging multiple, diverse representations with a router to better serve user intent.

5.1 The Map is Not the Territory

The Map Is Not The Territory

(And That's Good)

Dr. Bryan Bischof – Ayush Chaurasia



11/07/2025

(Timestamp: 01:00)

The presentation's central theme is that a “map” (a data representation) is not the same as the “territory” (the real-world data). In machine learning, this distinction is an advantage. Models and embeddings are our maps, and we can create many different maps of the same territory to serve different purposes.

5.2 Deconstructing RAG Buzzwords

The RAG landscape is filled with terms that can obscure fundamental principles. These terms can be deconstructed into simpler concepts.

Part I

“_____ RAG”

+ *Naive RAG*

(*Timestamp: 02:37*)

Naive RAG is better understood as **Simple RAG**: the foundational approach of searching a vector store with a vector to find similar items.

Part I

“_____ RAG”

- + *Naive RAG*
- + *Agentic RAG*

(*Timestamp: 04:04*)

Agentic RAG involves an LLM choosing *how* to search, giving the impression that the model can remove the engineer from designing the retrieval pipeline.

Part I

“_____ RAG”

- + *Naive RAG*
- + *Agentic RAG*
- + *Hybrid RAG*

(*Timestamp: 04:41*)

Hybrid RAG combines Simple RAG with classic retrieval techniques like keyword matching, allowing for searches with multiple signals simultaneously.

Part I

“_____ RAG”

- + *Naive RAG*
- + *Agentic RAG*
- + *Hybrid RAG*
- + *Graph RAG*

(*Timestamp: 05:05*)

Graph RAG uses the relationships *between* objects to improve retrieval, such as identifying stores that sell “home goods” to find a coffee filter, also considering proximity.

Part I

“_____ RAG”

- + *Naive RAG*
- + *Agentic RAG*
- + *Hybrid RAG*
- + *Graph RAG*
- + *Multi-Modal RAG*

(*Timestamp: 06:06*)

Multi-Modal RAG has two meanings: searching with multiple data types (text and images) or searching across multiple locations (“modes”) within the same latent space for a single item.

5.3 A First-Principles View of RAG

Advanced RAG techniques can be reframed as core engineering pipelines.

Part II



ONE

Hypothetical Document Embeddings with semantic search is a document enrichment pipeline, not a retrieval technique.

(*Timestamp: 07:58*)

Hypothetical Document Embeddings (HyDE) is a **document enrichment pipeline**. It uses an LLM to rewrite documents into the language that users are likely to search with. A dense, technical document can be rewritten into a simpler description, creating a new, more searchable “map” of the original.

Part II



ONE

Hypothetical Document Embeddings with semantic search is a document enrichment pipeline, not a retrieval technique.

TWO

Agentic RAG is a query enrichment pipeline, not a retrieval technique.

(*Timestamp: 10:14*)

Agentic RAG is a **query enrichment pipeline**. When a query is ambiguous, an agent decides how to search. For example, it determines whether “V60 filter” refers to a product or a restaurant, routing the query to the correct search process.

Part II



ONE

Hypothetical Document Embeddings with semantic search is a document enrichment pipeline, not a retrieval technique.

TWO

Agentic RAG is a query enrichment pipeline, not a retrieval technique.

THREE

Rank fusion (knn) is multi-stage processing, not a retrieval technique.

(*Timestamp: 11:38*)

Rank fusion is **multi-stage processing**. It involves running multiple different searches and then combining, or “stitching,” the results together in a subsequent stage.

5.4 The Three Responsibilities of an IR Engineer

These advanced techniques can be derived by focusing on three core responsibilities.

Part II

— You Could Have Invented —

The core responsibility of the IR engineer is to understand:

- What is the most likely representation of the user's desire?

(*Timestamp: 12:43*)

1. **Predict user intent:** What is the most likely representation of what the user is looking for?

Part II

— You Could Have Invented —

The core responsibility of the IR engineer is to understand:

- What is the most likely representation of the user's desire?
- What representations can you generate ahead of time from the entities?

(*Timestamp: 13:08*)

2. **Generate multiple representations (maps):** Create different views of the source data ahead of time (document enrichment).

Part II

— You Could Have Invented —

The core responsibility of the IR engineer is to understand:

- What is the most likely representation of the user's desire?
- What representations can you generate ahead of time from the entities?
- How can you correctly match these up?

(*Timestamp: 13:39*)

3. **Match intent to representation:** Correctly match the user's query with the appropriate pre-generated representation.

5.5 Practical Application: Curving Space

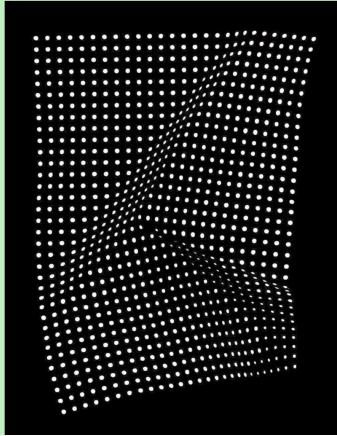
“Curving space” means shaping data representations and indices to improve search.

Part III

Let's retrieve Financial Documents

ONE

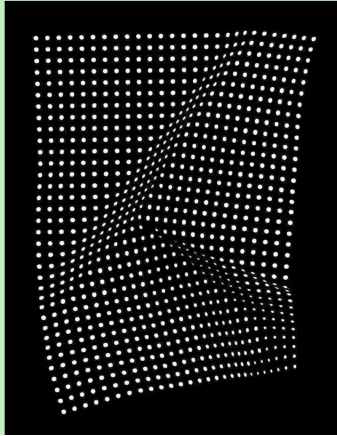
What are the different concepts one may construct from these docs? Summaries?
Tables of Data? Lists of Named Entities?
Form types/categories? What else?



(Timestamp: 13:58)

For financial documents, one could create multiple “maps” from a single corpus, such as summaries, tables of data, lists of named entities, and form types. This is document enrichment.

Part III



Let's retrieve Financial Documents

ONE

What are the different concepts one may construct from these docs? Summaries? Tables of Data? Lists of Named Entities? Form types/categories? What else?

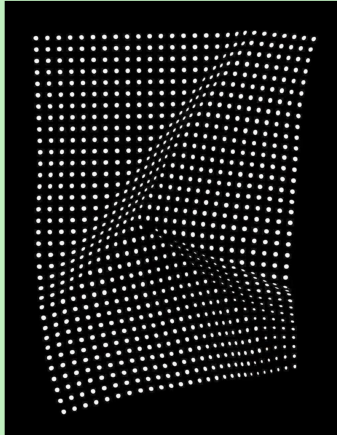
TWO

What is the appropriate index strategy for each? Semantic Search? BM25? Keyword Matching? Pre-filters? When should you use multiple on one index? When not?

(Timestamp: 16:34)

Once multiple representations exist, the right indexing strategy must be chosen for each. This involves deciding between semantic search, keyword matching, pre-filters, and whether to use a single index or separate ones.

Part III



Let's retrieve Financial Documents

ONE

What are the different concepts one may construct from these docs? Summaries? Tables of Data? Lists of Named Entities? Form types/categories? What else?

TWO

What is the appropriate index strategy for each? Semantic Search? BM25? Keyword Matching? Pre-filters? When should you use multiple on one index? When not?

THREE

When is another "stage" required? Is the easiest thing to retrieve what the agent needs? Can one retrieval enable another?

(Timestamp: 18:08)

Sometimes, a second retrieval step, informed by the first, is required. This is a form of staged processing.

5.6 Agents as Routers



(*Timestamp: 18:54*)

Agents are **transformers** in their function: they transform incoming data and instructions into a structured output.

Part IV

Agents Are Transformers

(no, not like that)

In both Agentic RAG and Multi-Agent RAG, the core idea is to use general LM's to transform incoming data and instructions into a structured output.

Thinking of them like 'routers' can be very helpful: route the data to different indices based on instructions, or route upstream stages to more retrieval stages.



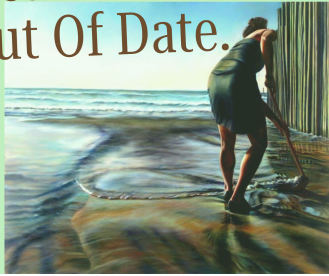
(*Timestamp: 19:04*)

An effective way to use agents is as **routers**. They take incoming data and route it to different indices or subsequent retrieval stages. This is the core idea behind both Agentic RAG and Multi-Agent RAG.

5.7 Dynamic Representations

Part V

The Danger In All Of This, Is That Many Documents Are Dynamic. This Means That The Representations Get Out Of Date.

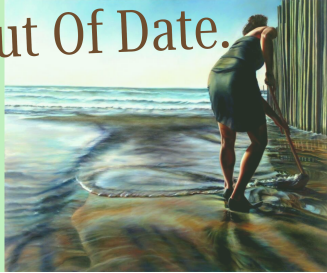


(Timestamp: 20:51)

A significant danger in RAG is that documents are dynamic. A static embedding will become outdated as the context of a document changes based on new business or world events.

Part V

The Danger In All Of This, Is That Many Documents Are Dynamic. This Means That The Representations Get Out Of Date.



The Upshot Here Is That You Need To Design The System To Shard Each Entity In The Easiest Way Possible To Understand What's Changed, And Then Re-Index That Which Has.

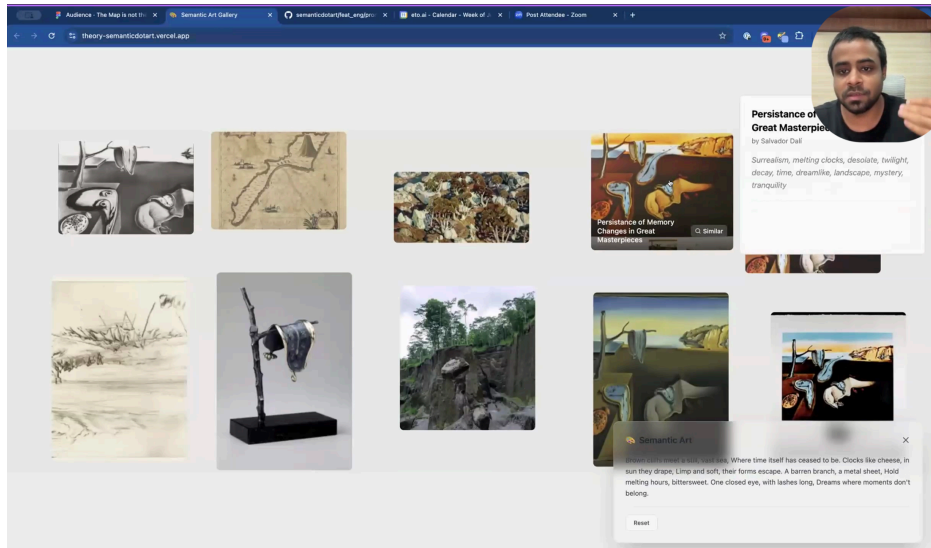
(*Timestamp: 22:49*)

The solution is to design a system that can detect what has changed and re-index only those parts, requiring an architecture built for dynamic updates.

5.8 Demo: Semantic Dot Art

(*Timestamp: 23:21*)

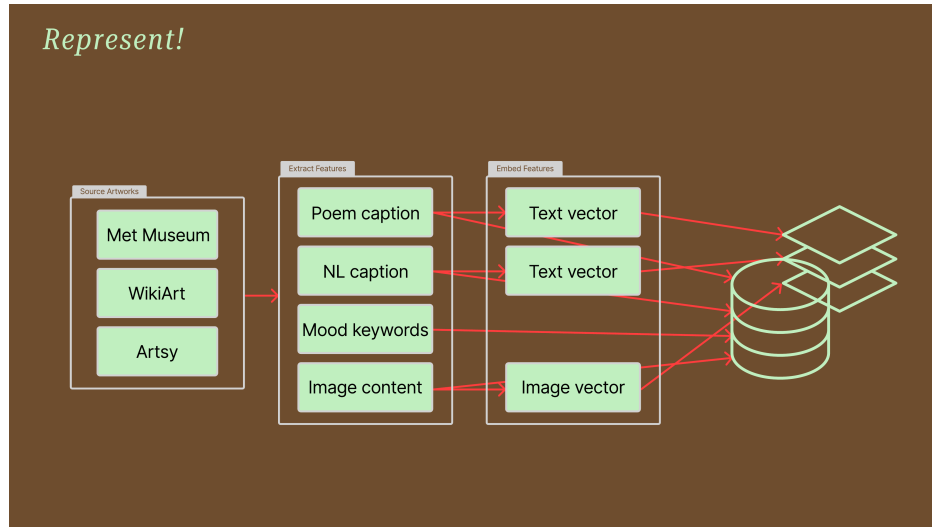
To make these concepts concrete, Ayush Chaurasia demonstrated **semanticdotart**, an application for discovering artworks. The demo shows how a system can serve diverse user intents by creating and searching over multiple representations of the same underlying data.



A user can search for art using different “maps” of the data. For example, they can search with a literal description (“multiple clocks melting in a desert”), a poetic description, or even a similar image. The system retrieves not only the original artwork but also derivative pieces and other thematically related works. This is made possible by a rich document enrichment pipeline that creates multiple vector and keyword-based indices for each artwork, capturing everything from mood and style to literal object descriptions.

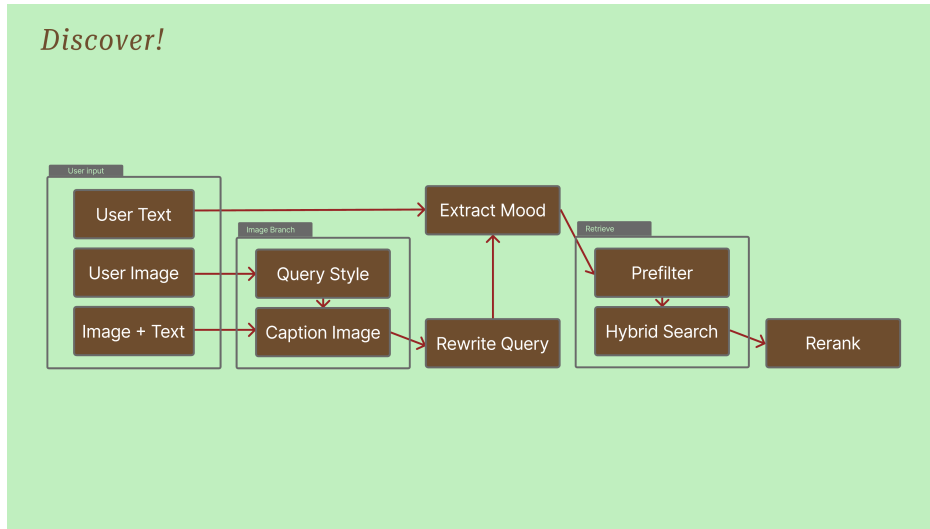
The retrieval process is agentic. The system routes the user’s query to the most appropriate index or combination of indices. A poetic query might be routed to an index of artistic descriptions, while an image query would use a multimodal embedding. This dynamic routing, combined with multi-stage processing and rank fusion, allows the system to handle a wide variety of user needs and deliver more satisfying, diverse results.

5.9 System Architecture



(*Timestamp: 27:26*)

The “Represent!” diagram visualizes the document enrichment pipeline. Data from various sources is processed to extract different features (poem captions, NL captions, mood keywords, image content). These are embedded into different vector types and stored, creating multiple “maps” for the same territory.



(*Timestamp: 29:10*)

The “Discover!” diagram shows the retrieval pipeline. User input is routed through various stages where it can be enriched (e.g., mood extraction, query rewriting). These enriched queries are then used in a multi-stage retrieval process involving pre-filtering and hybrid search before final reranking.

5.10 Integration with Other Techniques

The multiple representations approach naturally integrates with other advanced RAG techniques:

Reasoning Models are a form of query enrichment, rewriting confusing queries with necessary context.

Late-interaction models like ColBERT create diversity *within the model itself* by generating multiple representations (a vector for each

token) for a single document. This creates different “modes” in the latent space for more diverse and nuanced search results.

Routing is a retrieval stage where you decide which process or index to use based on the query. For a small number of routes, an LLM can act as a simple classifier. As the number of routes grows, a dedicated, trained classifier becomes more appropriate.

5.11 Chapter Principles

The key insights from this approach:

1. **No Single Perfect Map:** Instead of seeking one perfect representation, build multiple specialized representations of your data
2. **Intent-Based Routing:** Use agents or classifiers to route queries to the most appropriate representation
3. **Dynamic Updates:** Design systems that can detect changes and re-index incrementally
4. **Multi-Stage Processing:** Combine results from different representations and processing stages
5. **Domain Specialization:** Create representations tailored to specific types of queries or user intents

This framework provides a principled way to think about advanced RAG techniques as variations on the theme of multiple representations and intelligent routing.

5.12 Video

Here is the full video:

<https://youtu.be/hf9B3knU9vc>